

LINQ SelectMany Method in C# with Examples

In this article, I will discuss the **LINQ SelectMany Projection Method in C#** with Examples. Please read our previous article discussing the [LINQ Select Projection Operator in C#](#) with Examples. The LINQ SelectMany Method belongs to the projection Category Operator. As part of this article, we will discuss the following pointers in detail.

1. [What is LINQ SelectMany Method?](#)
2. [Example to Understand LINQ SelectMany Method using C#](#)
3. [LINQ SelectMany Using Query Syntax in C#](#)
4. [SelectMany Method with Complex Data Type in C#](#)
5. [How to Remove Duplicates while using LINQ SelectMany Method](#)
6. [Complex Example to Understand LINQ SelectMany Method using C#](#)
7. [When should we use the LINQ SelectMany Method in C#?](#)

What is LINQ SelectMany Method in C#?

The LINQ SelectMany Method in C# is used to project each element of a sequence or collection or data source to an `IEnumerable<T>` type and flatten the resulting sequences into one sequence. That means the SelectMany Projection Method combines the records from a sequence of results and then converts them into one result.

Imagine you have a list of objects, where each object contains a collection. You want to create a single, flat collection that contains all the elements from all the individual collections. SelectMany does exactly this. For example, if you have a list of three lists, where each inner list contains three numbers, SelectMany will give you a single list of nine numbers.

It is useful when working with collections of collections (e.g., lists of lists or arrays of arrays) or when you want to transform and combine elements from multiple sources into a single sequence. If this is not clear at the moment, don't worry; once we discuss a few examples, you will get the clarity. The syntax of the LINQ SelectMany Method is as follows:

```
IEnumerable<TResult> SelectMany<TSource, TResult>(
    IEnumerable<TSource> source,
    Func<TSource, IEnumerable<TResult>> selector
)
```

Here's what each parameter does:

1. **source**: This is the input sequence, typically a collection of elements.
2. **selector**: A function that takes an element from the input sequence and returns an `IEnumerable<TResult>`. This function is applied to each element in the source sequence, and the results are flattened into a single sequence.

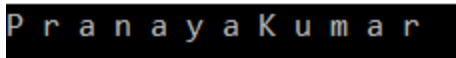
Example to Understand LINQ SelectMany Projection Method using C#:

Let us understand the LINQ SelectMany Method with an example using C#. In the example below, we use the SelectMany Method to flatten the resulting sequences into one sequence. Here, you can see that the SelectMany method returns an `IEnumerable<char>`. This is because the SelectMany method returns all the

elements from the sequence. Here, the nameList is the sequence or collection or the data source. The sequence contains two strings. And we know a string is a collection of characters. So, the SelectMany method fetches all the characters from the above two strings and converts them into one sequence, i.e., **IEnumerable<char>**.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<string> nameList =new List<string>(){ "Pranaya", "Kumar" };
            IEnumerable<char> methodSyntax = nameList.SelectMany(x => x);
            foreach(char c in methodSyntax)
            {
                Console.Write(c + " ");
            }
            Console.ReadKey();
        }
    }
}
```

So, when we execute the above program, it will give us the following output. It will combine all the characters of both strings and form the result as a collection of characters, as shown in the below output.



LINQ SelectMany Using Query Syntax in C#:

The most important point you need to remember is that no such SelectMany Operator is available in LINQ to write Query Syntax. But we can achieve this by writing multiple “from clauses” in the query, as shown in the below example. In the below example, we are fetching all the strings from the nameList collection to the str object. Then we use another from clause to fetch all the characters from the str object, and then we project the characters that will contain all the characters available in both strings.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<string> nameList =new List<string>(){ "Pranaya", "Kumar" };

            IEnumerable<char> querySyntax = from str in nameList
```

```

        from ch in str
        select ch;

    foreach (char c in querySyntax)
    {
        Console.Write(c + " ");
    }
    Console.ReadKey();
}
}
}

```

Now, execute the above example code, and you will see the same output as the LINQ Method Syntax, as shown in the below image.

P r a n a y a K u m a r

Example to Understand LINQ SelectMany Projection Method with Complex Data Type in C#

Create a class file named **Student.cs** and copy and paste the following code. As you can see, we have created the following Student class with four properties. Further, if you notice, the Programming property of the Student class returns **List<string>**. Here, we have also created one method to return the List of students, which will act as our data source.

```

using System.Collections.Generic;
namespace LINQDemo
{
    public class Student
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Email { get; set; }
        public List<string> Programming { get; set; }
        public static List<Student> GetStudents()
        {
            return new List<Student>()
            {
                new Student(){ID = 1, Name = "James", Email = "James@j.com", Programming = new List<string>() { "C#", "Java", "C++" } },
                new Student(){ID = 2, Name = "Sam", Email = "Sara@j.com", Programming = new List<string>() { "WCF", "SQL Server", "C#" } },
                new Student(){ID = 3, Name = "Patrik", Email = "Patrik@j.com", Programming = new List<string>() { "MVC", "Java", "LINQ" } },
                new Student(){ID = 4, Name = "Sara", Email = "Sara@j.com", Programming = new List<string>() { "ADO.NET", "C#", "LINQ" } }
            };
        }
    }
}

```

Now, our requirement is to Project all Programming strings of all the Students to a single **IEnumerable<string>**. As you can see, we have four students, so there will be 4 **IEnumerable<string>** sequences. And, we need to flatten to form a single sequence, i.e., a single **IEnumerable<string>** sequence.

```
Programming = new List<string>() { "C#", "Java", "C++" }
Programming = new List<string>() { "WCF", "SQL Server", "C#" }
Programming = new List<string>() { "MVC", "Java", "LINQ" }
Programming = new List<string>() { "ADO.NET", "C#", "LINQ" }
```

The complete example code is given below. In the below example, I am showing both LINQ Query Syntax and Method Syntax to achieve the same.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            List<string> MethodSyntax = Student.GetStudents().SelectMany(std =>
std.Programming).ToList();
            //Using Query Syntax
            IEnumerable<string> QuerySyntax = from std in Student.GetStudents()
                                             from program in std.Programming
                                             select program;
            //Printing the values
            foreach (string program in MethodSyntax)
            {
                Console.WriteLine(program);
            }
            Console.ReadKey();
        }
    }
}
```

When we execute the program, it will give us the following output. Here, you can see it contains all the programming strings from all the students. Also, if you notice, it contains the same string multiple times.

```
C#
Java
C++
WCF
SQL Server
C#
MVC
Java
LINQ
ADO.NET
C#
LINQ
```

Example to Understand How to Remove Duplicates While Using LINQ SelectMany Method:

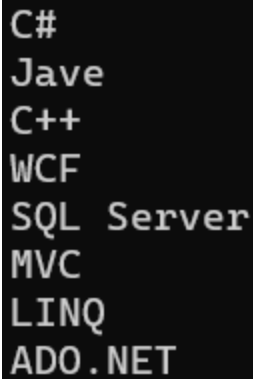
In our previous example, we get the output as expected but with duplicate program names. If you want only the distinct program names, then you need to apply the distinct method on the result set, as shown in the example below. The LINQ Distinct Method will return distinct elements from a sequence using the default equality comparer to compare values. In our upcoming articles, we will discuss Comparer and the Distinct Method in detail. For now, remember that the distinct method will return distinct elements from a sequence.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            List<string> MethodSyntax = Student.GetStudents()
                .SelectMany(std => std.Programming)
                .Distinct()
                .ToList();

            //Using Query Syntax
            IEnumerable<string> QuerySyntax = (from std in Student.GetStudents()
                from program in std.Programming
                select program).Distinct().ToList();

            //Printing the values
            foreach (string program in QuerySyntax)
            {
                Console.WriteLine(program);
            }
            Console.ReadKey();
        }
    }
}
```

With the above changes in place, now run the application code, and you will see that it will remove the duplicate program names and will show the distinct program names from the sequence, as shown in the image below.



```
C#
Java
C++
WCF
SQL Server
MVC
LINQ
ADO.NET
```

Complex Example to Understand LINQ SelectMany Method using C#:

Now, let us proceed and see more complex examples to understand the LINQ SelectMany Method. We want to retrieve the student's name and the program names. The following example exactly does the same. We will use the other overloaded version of the SelectMany method for this. In the below example, I am showing both Query and Method Syntax to achieve the same.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MethodSyntax = Student.GetStudents()
                .SelectMany(std => std.Programming,
                    (student, program) => new
                    {
                        StudentName = student.Name,
                        ProgramName = program
                    })
                .ToList();

            //Using Query Syntax
            var QuerySyntax = (from std in Student.GetStudents()
                               from program in std.Programming
                               select new {
                                   StudentName = std.Name,
                                   ProgramName = program
                               }).ToList();

            //Printing the values
            foreach (var item in QuerySyntax)
```

```

        {
            Console.WriteLine(item.StudentName + " => " + item.ProgramName);
        }
        Console.ReadKey();
    }
}

```

Output:

```

James => C#
James => Jave
James => C++
Sam => WCF
Sam => SQL Server
Sam => C#
Patrik => MVC
Patrik => Jave
Patrik => LINQ
Sara => ADO.NET
Sara => C#
Sara => LINQ

```

When should we use the LINQ SelectMany Method in C#?

Here are some common scenarios when you might want to use SelectMany:

- **Flattening Nested Collections:** If you have a collection of objects, each of which contains another collection, and you want to combine all the nested collections into a single flat collection, SelectMany is the right choice. For example, if you have a list of departments, and each department has a list of employees, and you want a combined list of all employees across all departments.
- **Cross Product or Cartesian Product:** When you need to perform a cross join or Cartesian product between two sequences where each item from the first sequence is paired with every item of the second sequence. This is often used in scenarios where you want to combine elements from different sources.
- **Querying Multi-Level Hierarchies:** In scenarios where you have multi-level nested structures, and you want to retrieve items from a deeper level directly. For example, accessing grandchild elements in a tree-like structure.
- **Concatenating Sequences Inside a Collection:** If you have a collection where each element itself is a sequence, and you want to concatenate these sequences into one single sequence.

In the next article, I will discuss the [Where Filtering Method in LINQ](#) with Examples. I hope this article gives you a good understanding of using the **LINQ SelectMany Method in C#** with Examples.